



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

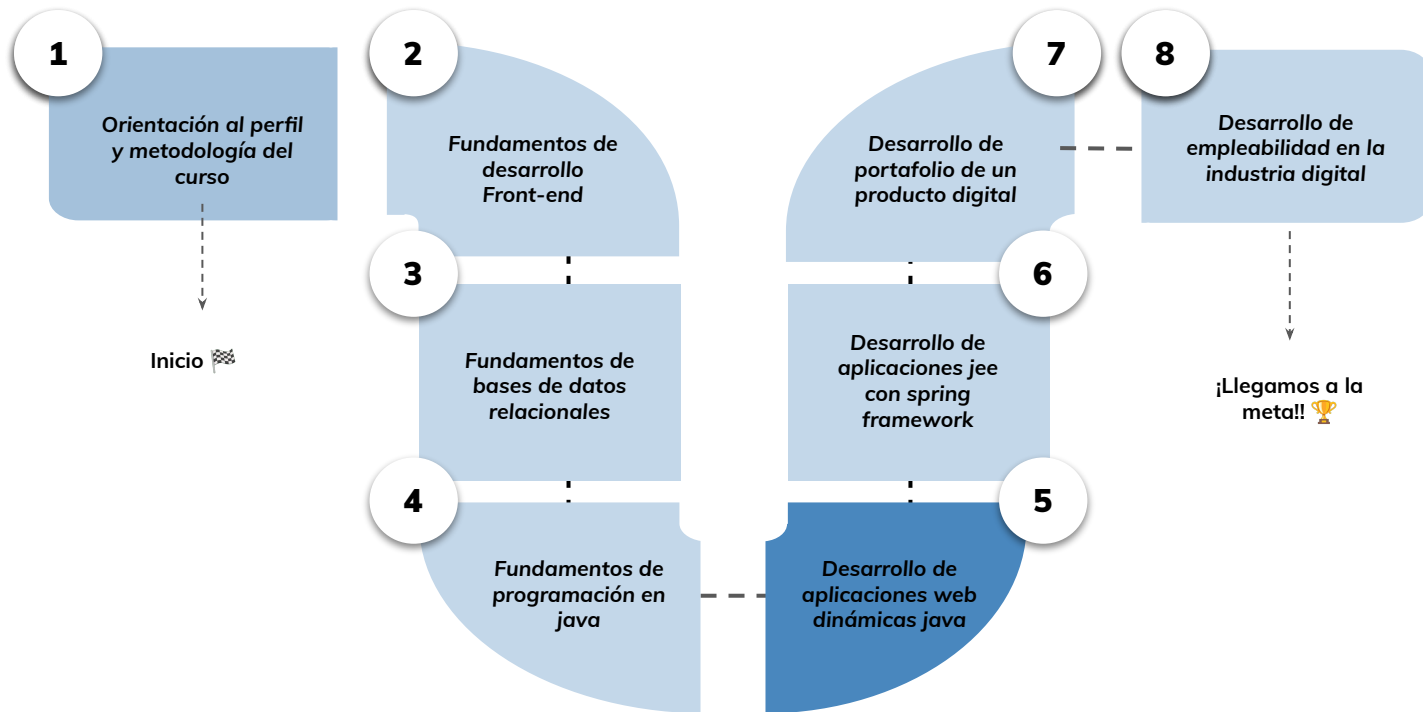


› El patrón de diseño MVC - Parte 2

Aprendizaje Esperado 5: Aplicar patrón modelo-vista- controlador en una aplicación web dinámica para la implementación de un flujo de navegación que despliega y captura información de acuerdo a lo solicitado.

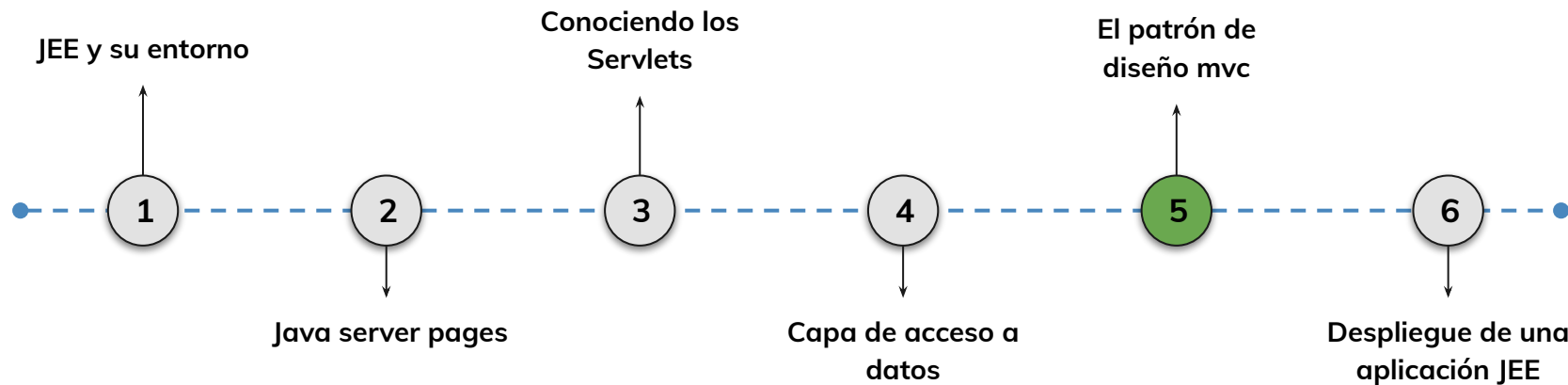
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

5.

Uso de frameworks modernos para implementar el patrón MVC

Exploraremos cómo frameworks como Spring MVC y JSF implementan el patrón Modelo-Vista-Controlador, y conoceremos sus ventajas, buenas prácticas, errores comunes y adaptaciones modernas del patrón.

Frameworks que aplican MVC en Java

Spring MVC, JSF y su estructura

Ejemplos prácticos con Spring MVC y JSF

Buenas prácticas y evolución del patrón MVC

Buenas prácticas y errores comunes en MVC

Variaciones, evolución moderna y cuándo aplicar el patrón

Objetivos de aprendizaje

¿Qué aprenderás?

- Identificar frameworks Java que implementan el patrón MVC.
- Analizar cómo Spring MVC y JSF aplican el modelo MVC.
- Reconocer errores comunes al implementar MVC.
- Aplicar buenas prácticas para separar correctamente las capas.
- Comprender la evolución del patrón MVC en arquitecturas modernas (SPAs, microservicios).

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Comprendimos qué es un patrón de diseño y su importancia en el desarrollo de software.
- Estudiamos la estructura del patrón Modelo-Vista-Controlador (MVC) y su separación de responsabilidades.
- Analizamos los componentes del modelo, la vista y el controlador en aplicaciones Java.
- Implementamos un ejemplo práctico de MVC utilizando Java EE con POJOs, Servlets y JSP.

➤ El patrón de diseño MVC



Frameworks MVC en Java



Spring MVC

Framework completo que implementa MVC con muchas funcionalidades adicionales.



Struts

Uno de los primeros frameworks MVC para Java, aún utilizado en muchos proyectos.



JavaServer Faces (JSF)

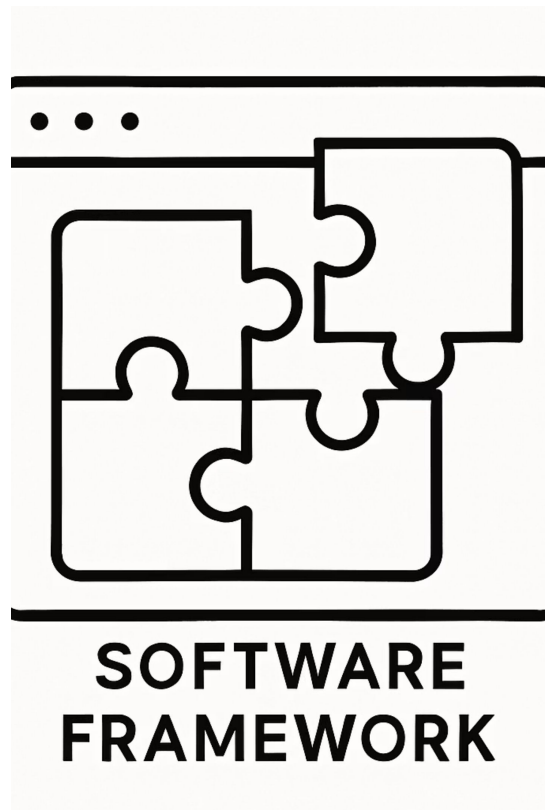
Framework oficial de Java EE para construir interfaces de usuario basadas en componentes.





Ventajas de usar frameworks MVC

- Implementan el patrón MVC de forma estandarizada
- Proporcionan herramientas adicionales (validación, seguridad, etc.)
- Reducen el código repetitivo (boilerplate)
- Facilitan el desarrollo de aplicaciones complejas
- Comunidad y documentación extensa





Implementación de MVC en Spring

Modelo

Clases de entidad con anotaciones JPA, repositorios y servicios.

Vista

Templates con Thymeleaf, JSP o motores similares.

Controlador

Clases anotadas con `@Controller` y métodos con `@RequestMapping`.



< > Ejemplo de Controlador en Spring

```
@Controller
@RequestMapping("/usuarios")
public class UsuarioController {

    @Autowired
    private UsuarioService usuarioService;

    @GetMapping("/lista")
    public String listarUsuarios(Model model) {
        List usuarios = usuarioService.listarTodos();
        model.addAttribute("usuarios", usuarios);
        return "usuarios/lista";
    }

    @GetMapping("/nuevo")
    public String formularioNuevo(Model model) {
        model.addAttribute("usuario", new Usuario());
        return "usuarios/formulario";
    }

    @PostMapping("/guardar")
    public String guardarUsuario(@ModelAttribute Usuario usuario) {
        usuarioService.guardar(usuario);
        return "redirect:/usuarios/lista";
    }
}
```

< > Ejemplo de Modelo en Spring MVC

```
// Entidad JPA
@Entity
@Table(name = "usuarios")
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String nombre;

    @Column(unique = true, nullable = false)
    private String email;

    @Column(nullable = false)
    private String password;

    // Getters y setters
}

// Repositorio
@Repository
public interface UsuarioRepository
    extends JpaRepository {

    Usuario findByEmail(String email);
}
```





Implementación de MVC en JSF

Modelo

Managed Beans con anotaciones @ManagedBean o CDI con @Named.

Vista

Páginas XHTML con componentes JSF y Expression Language.

Controlador

Managed Beans que manejan eventos y navegación.



< > Ejemplo de Managed Bean en JSF

```
@ManagedBean
@ViewScoped
public class UsuarioBean implements Serializable {

    private Usuario usuario = new Usuario();
    private List usuarios;

    @EJB
    private UsuarioService usuarioService;

    @PostConstruct
    public void init() {
        usuarios = usuarioService.listarTodos();
    }

    public String guardar() {
        usuarioService.guardar(usuario);
        usuario = new Usuario();
        return "lista?faces-redirect=true";
    }

    // Getters y setters
    public Usuario getUsuario() {
        return usuario;
    }

    public void setUsuario(Usuario usuario) {
        this.usuario = usuario;
    }

    public List getUsuarios() {
        return usuarios;
    }
}
```



< > Ejemplo de Vista en JSF

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
xmlns:h="http://java.sun.com/jsf/html"
xmlns:f="http://java.sun.com/jsf/core">
<h:head>
<title>Lista de Usuarios</title>
</h:head>
<h:body>
<h1>Usuarios Registrados</h1>

<h:dataTable value="#{usuarioBean.usuarios}" var="usuario">
<h:column>
<f:facet name="header">ID</f:facet>
#{usuario.id}
</h:column>

<h:column>
<f:facet name="header">Nombre</f:facet>
#{usuario.nombre}
</h:column>

<h:column>
<f:facet name="header">Email</f:facet>
#{usuario.email}
</h:column>

<h:column>
<f:facet name="header">Acciones</f:facet>
<h:link outcome="editar" value="Editar">
<f:param name="id" value="#{usuario.id}" />
</h:link>
</h:column>
</h:dataTable>
</h:body>
</html>
```





Buenas prácticas en MVC

Separación clara

Mantener una separación estricta entre las capas del MVC.

Modelo independiente

El modelo no debe depender de la vista ni del controlador.

Controlador ligero

El controlador debe delegar la lógica de negocio al modelo.

Vista pasiva

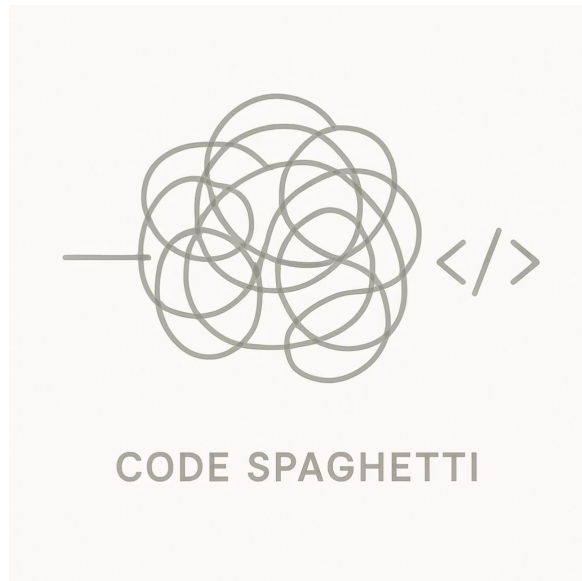
La vista solo debe mostrar datos, no procesarlos.





Errores comunes en MVC

- Mezclar lógica de negocio en la vista
- Colocar lógica de presentación en el modelo
- Controladores demasiado complejos
- Dependencias circulares entre capas
- No respetar la separación de responsabilidades





Variantes del patrón MVC

MVP

(Modelo-Vista-Presentador)

El presentador actúa como intermediario entre el modelo y la vista, con la vista siendo más pasiva.

MVVM

(Modelo-Vista-ViewModel)

Utiliza un ViewModel como capa intermedia que maneja el estado de la vista y se comunica con el modelo.

MVC con servicios

Añade una capa de servicios entre el controlador y el modelo para encapsular la lógica de negocio.





MVC en aplicaciones web modernas

En aplicaciones web modernas, el patrón MVC ha evolucionado:

- Backend como API (controlador y modelo)
- Frontend como SPA (vista)
- Comunicación mediante REST o GraphQL
- Frameworks JavaScript como React, Angular o Vue





MVC en microservicios

Modelo distribuido

El modelo puede estar distribuido en varios microservicios.

API Gateway

Un API Gateway puede actuar como controlador centralizado.

Vistas desacopladas

Las vistas pueden ser aplicaciones completamente separadas que consumen APIs.





Desafíos del MVC

- Complejidad inicial para proyectos pequeños
- Curva de aprendizaje para desarrolladores nuevos
- Posible sobrecarga de clases en proyectos grandes
- Mantenimiento de la separación de capas
- Adaptación a nuevos paradigmas (reactive, event-driven)





Cuándo usar MVC

Aplicaciones complejas

Ideal para aplicaciones con lógica de negocio compleja.

Equipos grandes

Facilita el trabajo en equipos donde diferentes personas se especializan en frontend y backend.

Proyectos a largo plazo

Beneficioso para proyectos que requieren mantenimiento a largo plazo.

Múltiples interfaces

Cuando se necesita soportar múltiples interfaces para los mismos datos.





Cuándo no usar MVC

- Aplicaciones muy simples o prototipos rápidos
- Proyectos pequeños con pocos desarrolladores
- Aplicaciones con poca o ninguna lógica de negocio
- Cuando el tiempo de desarrollo es extremadamente limitado
- Cuando otros patrones se adaptan mejor al problema





Alternativas a MVC

Arquitectura hexagonal

Enfocada en puertos y adaptadores para desacoplar la lógica de negocio.

Arquitectura basada en eventos

Centrada en la comunicación mediante eventos entre componentes desacoplados.

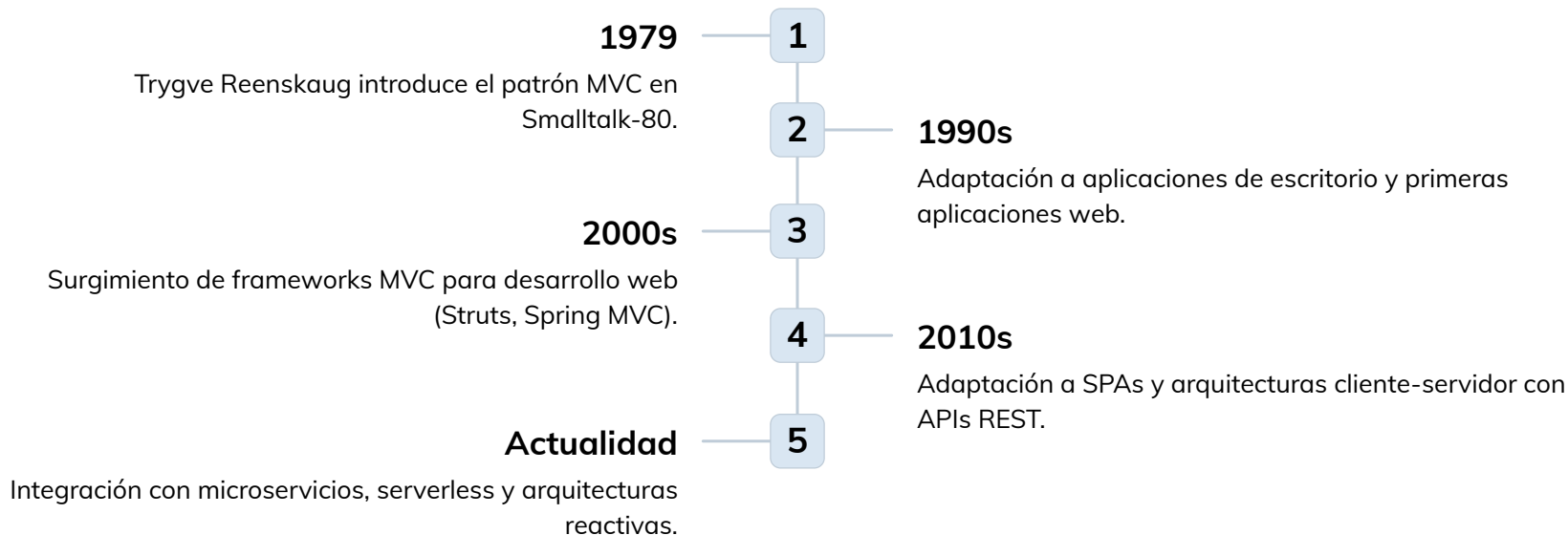
Arquitectura orientada a servicios

Organiza la aplicación como un conjunto de servicios independientes.





Evolución del MVC





Futuro del MVC

El patrón MVC sigue evolucionando con nuevas tendencias:

- Integración con paradigmas reactivos y funcionales
- Adaptación a arquitecturas serverless
- Uso en aplicaciones móviles y multiplataforma
- Combinación con otros patrones arquitectónicos





Resumen: Modelo

Responsabilidades

- Maneja datos y lógica de negocio
- Encapsula el estado de la aplicación
- Independiente de la interfaz de usuario

Componentes

- Entidades (POJOs)
- DAOs y Repositorios
- Servicios de negocio
- Validadores





Resumen: Vista

Responsabilidades

- Presenta la información al usuario
- Captura interacciones del usuario
- No contiene lógica de negocio

Componentes

- Páginas JSP/HTML
- Templates
- Componentes visuales
- Estilos y scripts





Resumen: Controlador

Responsabilidades

- Procesa peticiones del usuario
- Coordina modelo y vista
- Maneja el flujo de la aplicación

Componentes

- Servlets
- Filtros
- Controladores
- Manejadores de eventos





Beneficios clave del MVC

Separación de conceptos

Clara división de responsabilidades entre capas



Trabajo en equipo

Permite división de tareas entre especialistas



Reutilización

Componentes reutilizables en diferentes contextos



Mantenibilidad

Facilita el mantenimiento y evolución del código





Consejos para implementar MVC

1

Planificar la arquitectura

Diseñar la estructura de la aplicación antes de comenzar a codificar.

2

Usar frameworks

Aprovechar frameworks que implementan MVC para ahorrar tiempo.

3

Mantener la separación

Ser estricto con la separación de responsabilidades entre capas.

4

Documentar

Documentar la arquitectura y las decisiones de diseño.





Recursos para aprender más

Libros recomendados

- "Patterns of Enterprise Application Architecture" - Martin Fowler
- "Head First Design Patterns" - Freeman & Robson
- "Pro Spring MVC" - Deinum, Serneels, et al.

Recursos online

- Documentación oficial de Spring MVC
- Tutoriales de Java EE en Oracle
- Cursos en plataformas como Udemy, Coursera





Preguntas frecuentes

¿MVC es solo para aplicaciones web?

No, aunque es muy popular en desarrollo web, MVC puede aplicarse a aplicaciones de escritorio, móviles y otros tipos de software.

¿Cuál es la diferencia entre MVC y MVP?

En MVP, el Presentador tiene más control sobre la Vista que en MVC, donde el Controlador no necesariamente conoce la implementación de la Vista.

¿Es MVC adecuado para aplicaciones pequeñas?

Puede ser excesivo para aplicaciones muy simples, pero incluso proyectos pequeños pueden beneficiarse de la separación de conceptos si se espera que crezcan.

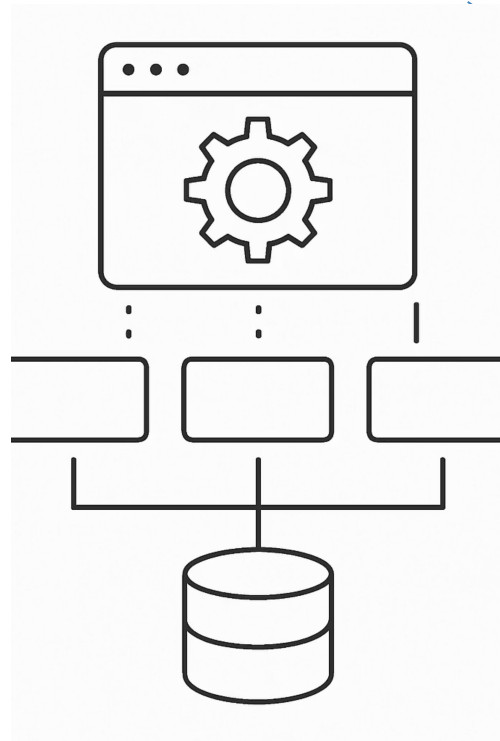


Conclusión

El patrón MVC es una herramienta poderosa para estructurar aplicaciones web con Java EE, proporcionando:

- Separación clara de responsabilidades
- Mayor mantenibilidad y escalabilidad
- Facilidad para el trabajo en equipo
- Reutilización de componentes

Dominar este patrón te permitirá crear aplicaciones web robustas y bien estructuradas.



Live Coding

¿En qué consistirá la Demo?

Crearemos una mini aplicación web usando Spring Boot y Thymeleaf, aplicando el patrón MVC. Desarrollaremos el flujo completo de registro y visualización de usuarios.

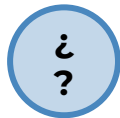
1. Crear proyecto Spring Boot con dependencias de Spring Web y JPA
2. Definir la entidad Usuario con anotaciones JPA
3. Crear UsuarioRepository como interfaz DAO
4. Implementar UsuarioService con la lógica de negocio
5. Desarrollar el controlador UsuarioController con métodos para listar y guardar
6. Crear vistas Thymeleaf para formulario y listado
7. Probar flujo de navegación desde el navegador
8. Mostrar validaciones y redirecciones funcionando correctamente

Tiempo: 25 minutos

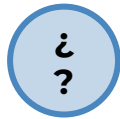
#Momentode Preguntas...



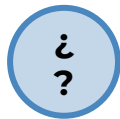
¿Qué diferencias principales existen entre Spring MVC y JSF?



¿Qué errores comunes debemos evitar al aplicar MVC?



¿Cuándo conviene usar un patrón alternativo como MVVM o arquitectura hexagonal?



¿Cómo se adapta MVC a proyectos modernos con frontend desacoplado y APIs?



Momento:

Time-out!



5 -10 min.





Ejercicio N° 1

Registro de usuarios con Spring MVC

Registro de usuarios con Spring MVC

Contexto: 🙌

Un equipo está desarrollando un sistema de gestión de usuarios utilizando el framework Spring MVC. Necesitan implementar un flujo que permita registrar nuevos usuarios, validar la información y mostrar la lista actualizada.

Consigna: 📝

Crea una aplicación con Spring Boot que implemente el patrón MVC. Debe incluir un formulario para crear usuarios, almacenar la información y mostrar una lista en pantalla utilizando Thymeleaf.

Tiempo 🕒: 45 minutos

Paso a paso: ⚙️

1. Crea una clase Usuario anotada con @Entity
2. Define el repositorio UsuarioRepository que extienda JpaRepository
3. Implementa UsuarioService para la lógica de negocio
4. Desarrolla el controlador UsuarioController con métodos @GetMapping y @PostMapping
5. Crea las vistas HTML con Thymeleaf para el formulario y el listado
6. Configura application.properties y el puerto de tu app
7. Ejecuta la app y valida el flujo de navegación
8. Prueba los casos de uso y captura resultados

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ Aprendimos sobre frameworks que aplican MVC en Java
- ✓ Implementamos el patrón con Spring MVC y JSF
- ✓ Aplicamos buenas prácticas para mantener separadas las capas
- ✓ Exploramos variantes como MVP y MVVM
- ✓ Conocimos la evolución del MVC hacia microservicios y SPAs
- ✓ Discutimos cuándo aplicar o evitar MVC según el caso



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son claves para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compártela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

